

Pet Pose Perception — Preliminary Engineering Task

Goal. Build a lightweight CPU-friendly perception pipeline that maps dog observations to sitting, standing, lying, or unknown while reducing frame-to-frame flicker in continuous video.

1. Architecture Choice

The supplied dataset contains 150 training images and 30 fixed test images, organised by pose. Each YOLO label provides a dog bounding box and pose/keypoint annotations. The deployed classifier uses a compact appearance representation: local HOG descriptors plus a 24×24 grayscale appearance descriptor and six normalized bounding-box geometry values. An Extra-Trees classifier was selected because it is fast on CPU, works well on small tabular feature sets, and requires no GPU at inference time.

For video, the script uses a lightweight motion-based ROI proposal with a large center-crop fallback. This avoids depending on an external heavyweight detector or downloading additional neural-network weights in the grading environment.

2. Open-Set and Temporal Handling

The classifier does not blindly force every frame into a known pose. A prediction becomes **unknown** when its confidence is below 0.52 or when the difference between the best and second-best class probabilities is below 0.12. A seven-frame history with a three-vote persistence rule reduces isolated classification flicker. These choices are deliberately conservative for a robotics setting where an uncertain state is preferable to a confident incorrect transition.

3. Test Results

Class	Precision	Recall	F1
Sitting	0.5556	0.5000	0.5263
Standing	0.6667	0.8000	0.7273
Lying	0.8889	0.8000	0.8421
Macro avg	0.7037	0.7000	0.6986

Overall accuracy: 70.00% (21/30). Confusion matrix (rows = ground truth; columns = predicted): sitting → [5,4,1], standing → [2,8,0], lying → [2,0,8].

Failure analysis. Sitting is the weakest class. Several sitting examples are visually close to standing or lying because body orientation, occlusion and background framing change the apparent silhouette. Standing and lying are more separable in this small test set. The fixed test set is only 30 images, so the metric should be interpreted as an indicative benchmark rather than a production accuracy estimate.

4. Compute Efficiency and Reproducibility

The implementation avoids a large deep-learning detector during grading. Feature extraction uses OpenCV/scikit-image and the Extra-Trees model is CPU-native. Dependencies are pinned for a clean Python 3.10 environment. The required video interface is exactly: `python inference_video.py --video <video> --weights <model.pt> --output <predictions.csv>`.

5. Limitations and Future Work

The dataset is intentionally small, and the current ROI proposal assumes a prominent/mostly central dog. For a production robot, the next improvement would be a small pretrained dog detector or pose estimator, followed by landmark-based pose classification and confidence calibration on real transition videos. A larger video dataset would also allow better temporal modelling and a more reliable unknown-state threshold.